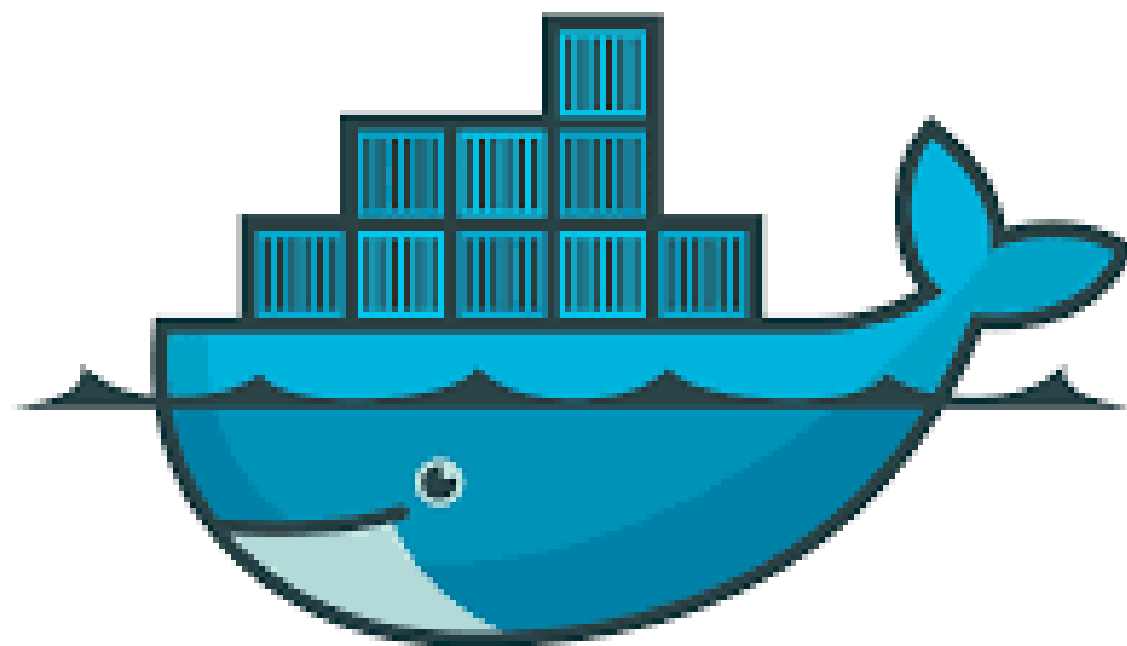




docker

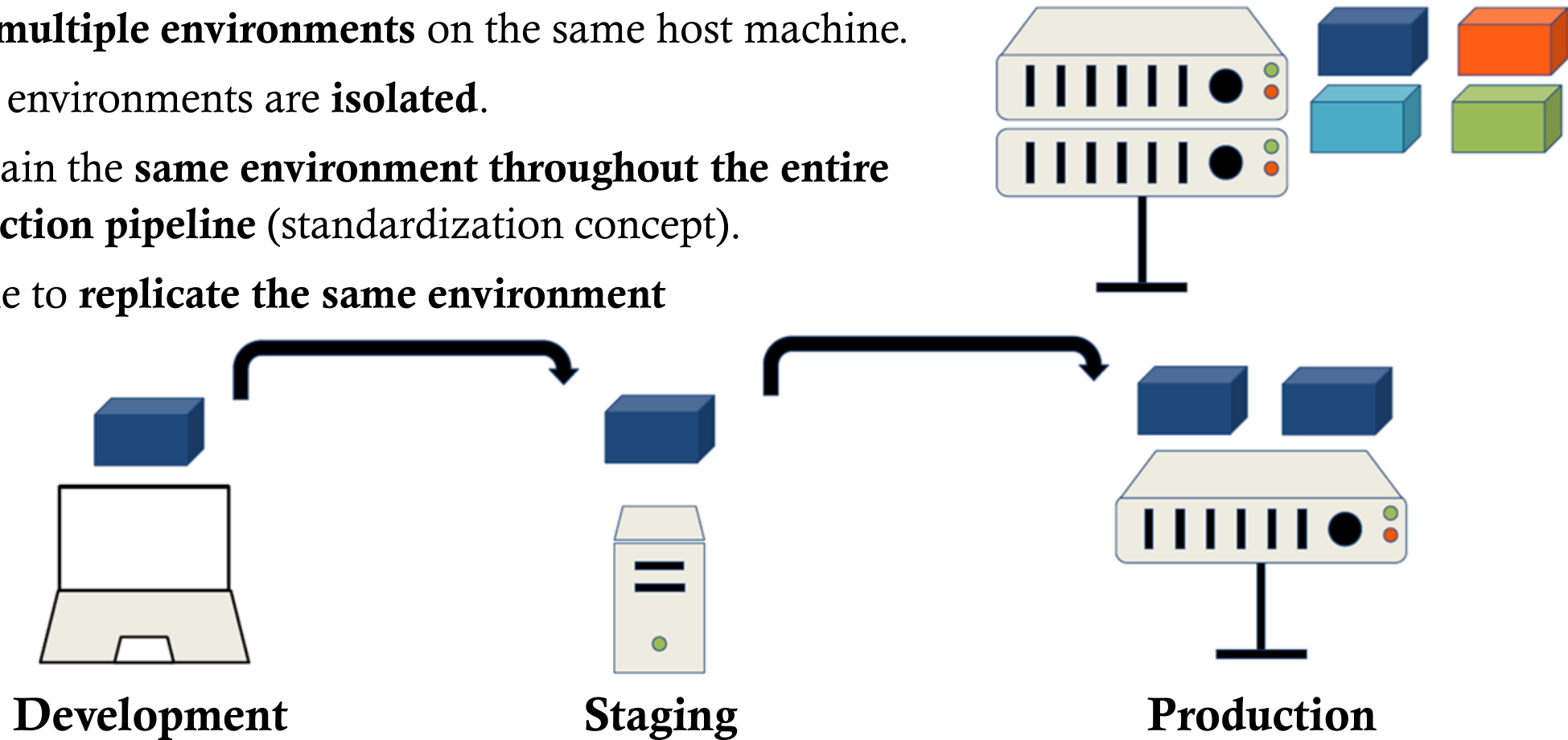
INTRODUCTION TO DOCKER



INTRODUCTION TO DOCKER

Environment Management

- Have **multiple environments** on the same host machine.
- These environments are **isolated**.
- Maintain the **same environment throughout the entire production pipeline** (standardization concept).
- Be able to **replicate the same environment**



INTRODUCTION TO DOCKER

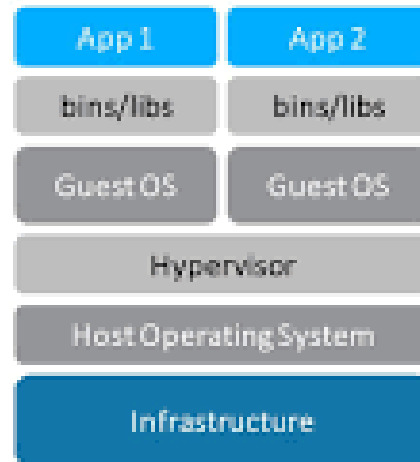
Virtualization and containerization

Virtual machines

A **virtual machine (VM)** hosts a complete environment including:

- its own **operating system**
- its own **libraries and binaries**
- its own **applications**

Each virtual machine is managed by a **hypervisor**.

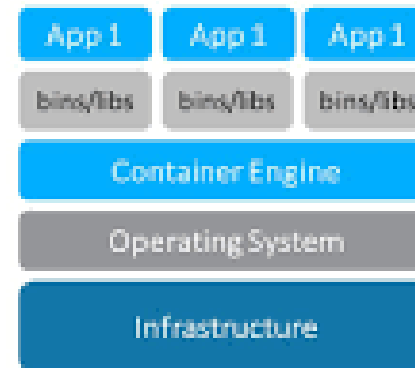


Virtual Machines

Containers

A **container** does not include a full operating system. Instead, containers:

- share the **host machine kernel**
- package only the **libraries, binaries, and dependencies** required by the application
- run through a container runtime such as **Docker Engine**



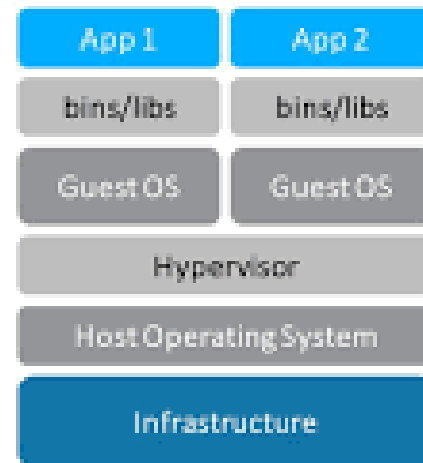
Containers

INTRODUCTION TO DOCKER

Virtualization and containerization

Virtual machines

Key idea: each VM contains a full guest OS, which makes virtualization powerful but generally heavier in terms of resources and startup time.

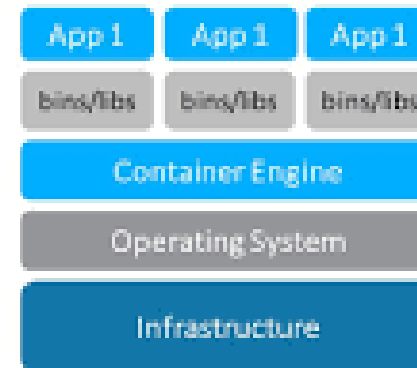


Virtual Machines

Containers

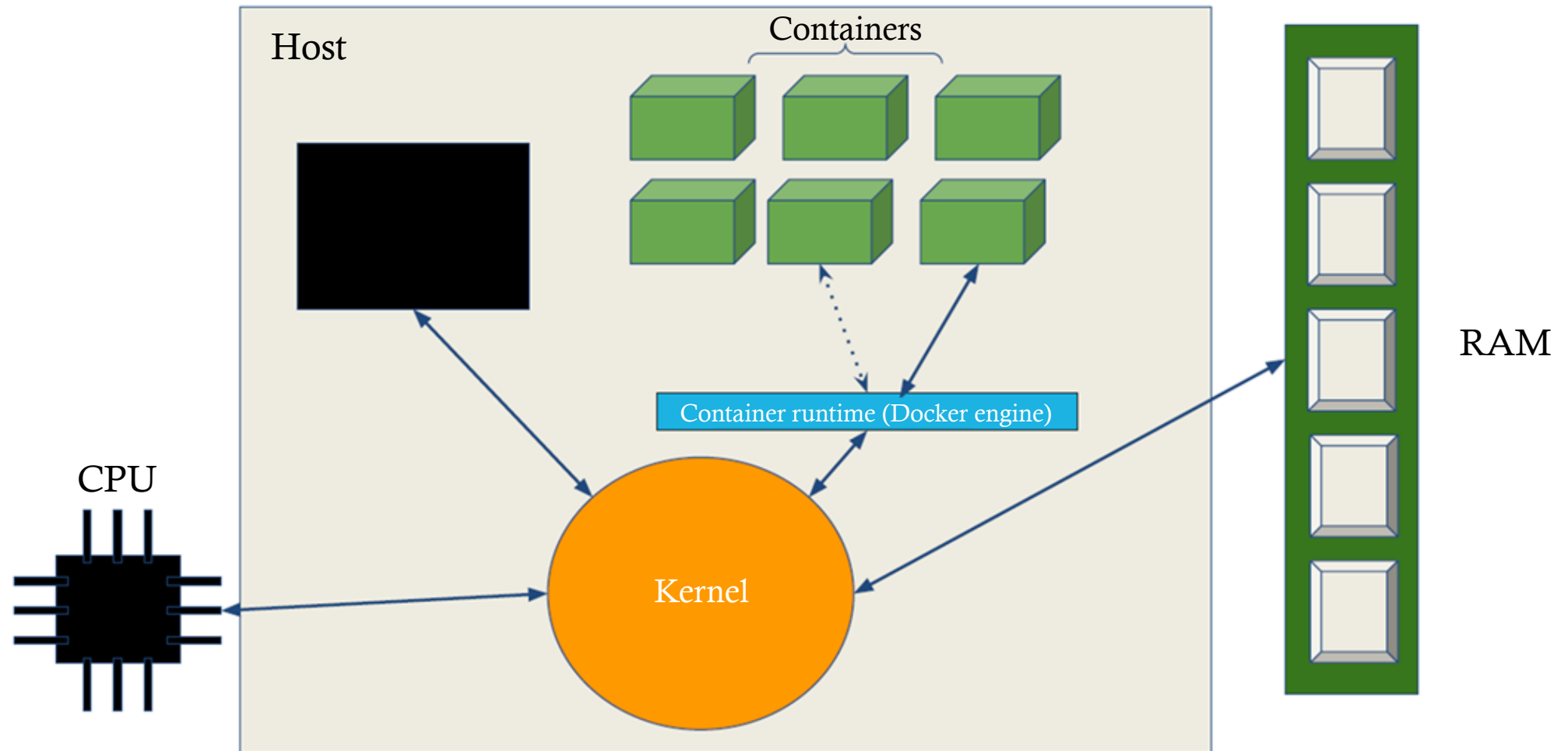
Key idea: containers are usually:

- lighter
 - faster to start
 - easier to replicate and deploy consistently
- because they avoid duplicating a complete OS for each application.



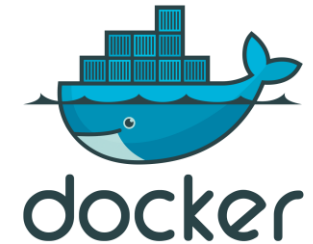
Containers

INTRODUCTION TO DOCKER



INTRODUCTION TO DOCKER

What is Docker?



Docker is a container platform that makes applications portable by packaging them with their dependencies and running them as isolated processes on a shared operating system.

INTRODUCTION TO DOCKER

Docker on Linux/Windows

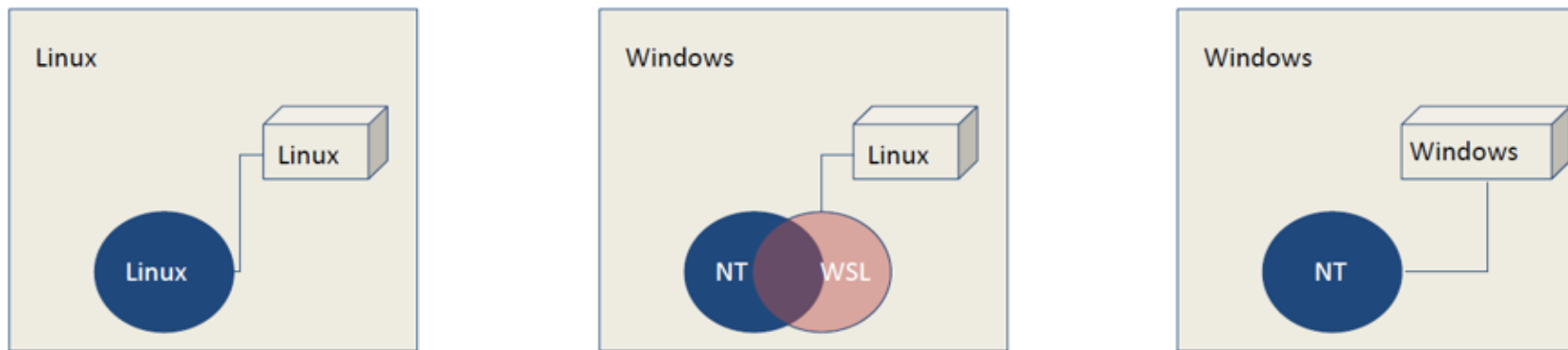
Containers rely on the host operating system's kernel to run.

A Linux container shares the host's Linux kernel.

So, how does Docker work on Windows?

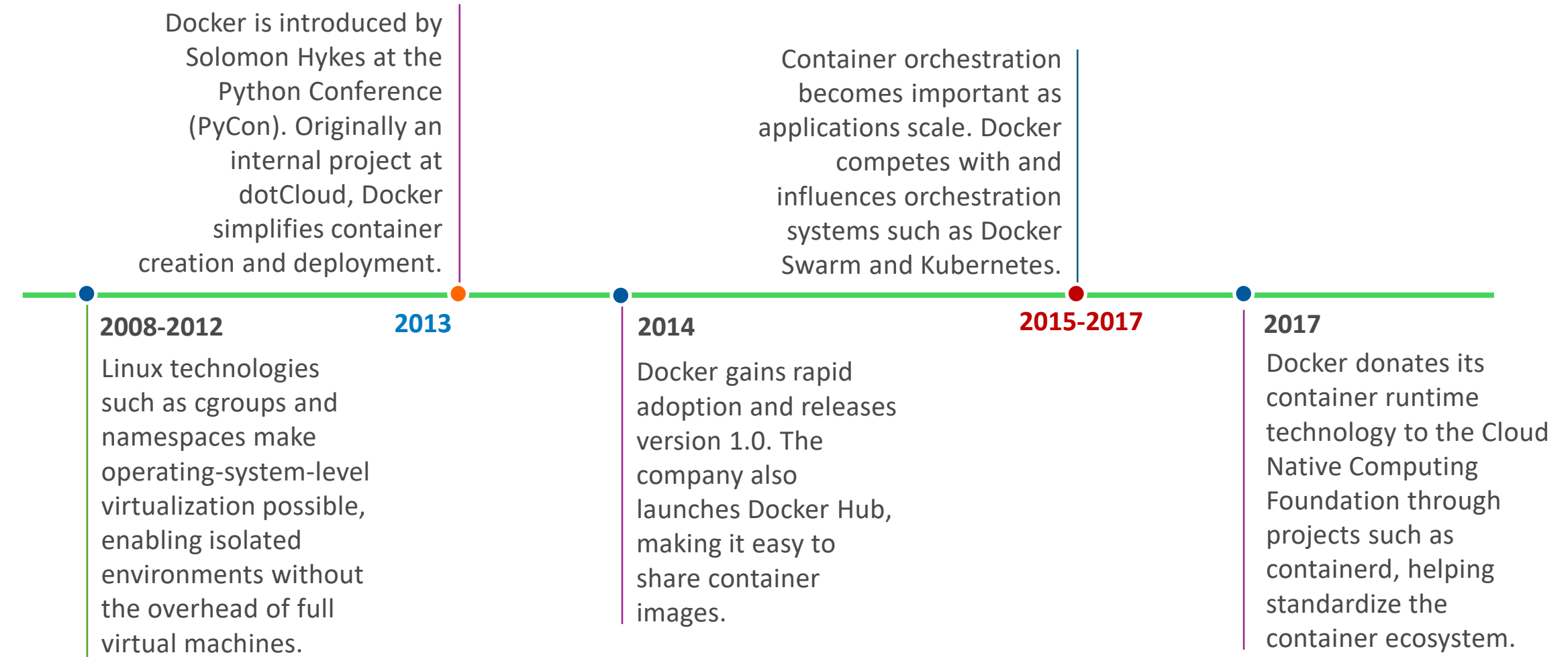
Docker can run two different types of containers on Windows:

- A Windows container packages a Windows user-space environment and runs directly on the Windows NT kernel.
- Most developers use Linux containers on Windows through Windows Subsystem for Linux (WSL 2).



INTRODUCTION TO DOCKER

A brief history



INTRODUCTION TO DOCKER

Who is it intended for?

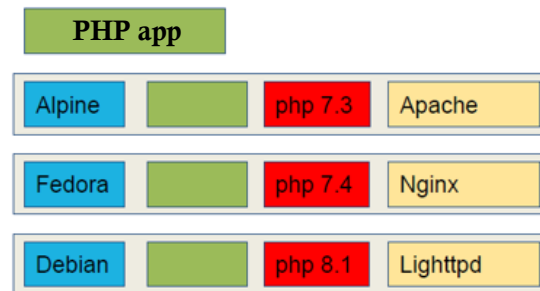
Developers

Using Docker, developers can package each application together with its runtime, dependencies, and configuration.

Different applications may require different technology stacks. For example, one application may use PHP 7.3 and Apache, while another uses PHP 8.1 and Nginx. Docker allows these applications to run in isolated containers without conflicts.

It also enables developers to test the same application across multiple environments, ensuring consistent behavior from development to production.

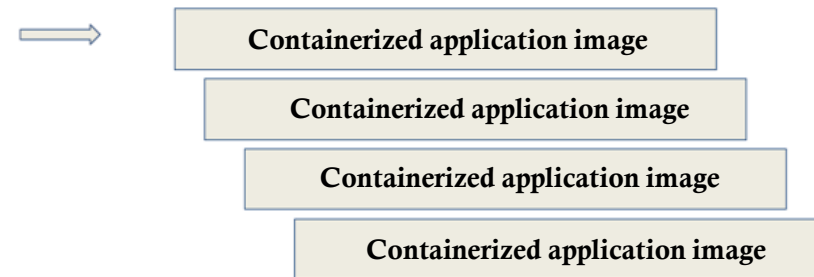
Example: A PHP application can be containerized with a specific Linux distribution, PHP version, and web server, creating a portable and reproducible environment.



System admins / Operations

Once the application has been packaged as a Docker image, it can be deployed repeatedly across development, testing, and production environments.

Administrators no longer need to manually configure each server. The same image can be deployed anywhere, ensuring consistency, simplifying updates, and making applications easier to monitor and maintain.



INTRODUCTION TO DOCKER

Benefits

For system admins

Isolation

Containers provide process, filesystem, and network isolation from the host system and from each other.

Each container runs in a controlled environment, reducing the impact of a compromised application on the host or other containers.

However, containers share the host operating system kernel, so isolation is not as strong as with virtual machines.

Multiple versions

Containers make it possible to run different applications, or different versions of the same software, on the same host without conflicts.

For example:

- one application may require PHP 5.x,
- another may require PHP 7.x,
- different services may also depend on different versions of databases such as MySQL.

Each container includes its own dependencies and runtime environment.

INTRODUCTION TO DOCKER

Benefits

For system admins

Cost efficiency

A single physical or virtual machine can be divided into multiple lightweight containers.

This allows better utilization of CPU and memory resources compared to running separate virtual machines for each application.

Resource limits (CPU, memory, etc.) can be enforced using kernel-level controls such as cgroups.

Scalability

Containers can be easily replicated to handle changes in workload.

With container orchestration platforms such as Kubernetes, applications can be automatically scaled up or down, load-balanced, and restarted if they fail.

INTRODUCTION TO DOCKER

Benefits

For developers

One of the classic problems in software development is: *“It works on my machine.”*

Docker solves this by standardizing environments. Instead of each developer (or even the client) running slightly different setups, everyone works from the same container image. This ensures that the application behaves consistently across all stages (development, testing, and production).

INTRODUCTION TO DOCKER

Benefits

For developers

CI/CD improvements

Docker...:

- ... enables automated testing in clean and reproducible environments. Tests can be executed on isolated containers in CI pipelines, ensuring that they run in an environment as close as possible to production. This avoids issues caused by polluted local environments and reduces inconsistencies between machines.
- ... simplifies deployment. Since your application is packaged inside a Docker image, there is no need to manually configure the production server with specific dependencies or runtime versions. Whether you deploy to a physical server, a VM, or a cloud platform, you simply run the container. Everything needed for the application is already included in the image, which makes deployments more predictable and portable.

INTRODUCTION TO DOCKER

Benefits

For developers

Working with multiple versions

- Docker is also very useful when dealing with multiple versions of software or frameworks.
- For example, in a long-lived project, older versions of your application may still be in production while new versions are being developed. Docker allows you to maintain and test fixes on legacy versions without interfering with your current development environment.
- It is also helpful when preparing for future upgrades. If a new version of a framework is released, you can safely test it in an isolated container environment without affecting your main system. This makes it easier to experiment with beta versions or migrations while keeping your local setup stable.

INTRODUCTION TO DOCKER

Terminology

- **Container**: a lightweight, isolated runtime environment that contains your application and everything it needs to run (code, dependencies, libraries). You can think of it as a “mini computer” running your app.
- **Docker image**: a blueprint used to create containers. It is a read-only template that includes your application code, runtime, system tools, and dependencies. When you run an image, it becomes a container.
- **Dockerfile**: a text file where you define how to build an image. It contains step-by-step instructions like which base image to use, what dependencies to install, and what command should start your app.
- **Docker daemon**: the background service that actually builds, runs, and manages containers. When you run Docker commands, you are communicating with this daemon.
- **Docker client**: the command-line tool (docker) used to interact with Docker. It sends instructions to the daemon.
- **Registry**: where images are stored and shared. The most common one is Docker Hub, but companies often use private registries as well.

INTRODUCTION TO DOCKER

Terminology

- **Volume**: a way to persist data outside of a container. Since containers are ephemeral (they can be deleted and recreated), volumes ensure your data is not lost.
- **Network**: in Docker, allows containers to communicate with each other (for example, a backend talking to a database container).
- **Container orchestration systems**: used when you need to manage many containers across multiple machines, handling scaling, deployment, and reliability (ex.: Kubernetes).
- **Stateless containers**: containers that do not keep any data internally between runs. If the container stops or is deleted, nothing important is lost inside it. Every request is independent. You can scale them easily (just spin up more identical containers). Typical examples include web servers, APIs, frontend services, microservices that call external databases. This is the “ideal” model for Docker, because containers are meant to be ephemeral (temporary and replaceable).
- **Stateful containers**: containers that store data locally inside the container. If the container is removed, data can be lost (unless properly handled). Scaling is more complex because each instance may have different data. Typical examples include databases (PostgreSQL, MySQL, Redis) and file storage systems. In Docker, statefulness is usually handled **using volumes (or external storage systems)**, so that the data lives outside the container itself.

IMAGE AND CONTAINER MANAGEMENT

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

In this section, try running some of the commands.

You can use an interactive shell and a long running-service.

```
docker run -it ubuntu bash
```

```
docker run nginx
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker run

Create and start a container from an image.

```
docker run hello-world
```

↑
image



Guess what these commands do (and try to run some of them):

```
docker run -it ubuntu bash
```

```
docker run -p 8080:80 nginx
```

```
docker run -d nginx
```

```
docker run --name my-app nginx
```

```
docker run -v $(pwd):/app my-app
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker run

-i = interactive (keep input open)

-t = terminal (gives you a shell)

-p maps ports. In our example 8080:80, 8080 is your machine, 80 is inside container, so you can open <http://localhost:8080>.

-d = detached (runs in background)

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

`docker ps`

Show all currently running containers on your system.

- CONTAINER ID: unique identifier of the container
- IMAGE: which image it was created from
- COMMAND: command running inside the container
- STATUS: running time or state
- PORTS: exposed ports
- NAMES: container name (auto-generated or user-defined)

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker ps

`docker ps -a` : show all containers, including stopped, exited and crashed ones.

`docker ps -q` : show only container IDs (useful for scripting, for example stopping all containers).

`docker ps --filter "status=exited"` : filter containers (by status, name, ancestor image).

`docker ps --format "{{.Names}}: {{.Status}}"` : format outputs.

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker logs

Used to view the output (stdout and stderr) of a container.

One of the first commands you will use when troubleshooting a container that is not behaving as expected.

-f = follow: follow logs in real time

--tail = show only the last lines `docker logs --tail 50 my-app`

-t = include timestamps

Usually, logs should be written to the console rather than to files inside the container.

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker stop

Gracefully stop a running container.

Docker accepts either the container name or ID.

```
docker stop my-app
```

```
docker stop 7f3c2a1b9d4e
```

After `docker stop`, the container still exists and can be restarted.

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

`docker stop`

What happens internally?

Docker:

1. Sends a SIGTERM signal to the main process inside the container.
2. Waits for the process to shut down cleanly (default: 10 seconds on Linux).
3. If the process is still running, Docker sends SIGKILL to force termination.

This gives the application a chance to finish ongoing requests, close database connections, flush logs, save state...

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker stop

Stop multiple containers:

```
docker stop container1 container2 container3
```

```
docker stop $(docker ps -q)
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker start

Start an existing stopped container.

```
docker start container1 container2 container3
```

```
docker start $(docker ps -q)
```

Unlike `docker run`, which creates a new container and starts it, `docker start` simply starts a container that already exists.

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker restart

Equivalent to:

```
docker stop my-app  
docker start my-app
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

`docker rm`

Remove one or more containers from your system.

With the basic command, you cannot remove a running container.

```
docker stop my-app  
docker rm my-app
```

or

```
docker rm -f my-app
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

`docker rm`

Very common cleanup command:

```
docker container prune
```

or

```
docker rm $(docker ps -a -q)
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker images

List the Docker images stored on your local machine.

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	24.04	abc123def456	2 weeks ago	77MB
nginx	latest	789xyz654abc	1 month ago	192MB

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker images

Show all images, including intermediate layers: `docker images -a`

Show only image IDs: `docker images -q`

Filter by repository name: `docker images nginx`

Display images with a custom format:

```
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker pull

Download a Docker image from a registry (by default, Docker Hub) to your local machine.

```
docker pull <image>:<tag>
```

What happens when you run it?

Docker checks if the image already exists locally. If not (or if a newer version is requested), it downloads image layers and metadata, then it stores it in your local image cache

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker pull

Download a Docker image from a registry (by default, Docker Hub) to your local machine.

```
docker pull <image>:<tag>
```

Always pull newer version (ignore cache): `docker pull --pull`

Quiet mode (less output): `docker pull -q nginx`

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker build

Create a Docker image from a Dockerfile. It takes the instructions in a Dockerfile and turns them into a runnable image.

```
docker build -t <image-name>:<tag> <path>
```

-t names (tags) the image

<path> directory containing the Dockerfile (often .)

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker build

No cache (force rebuild everything): `docker build --no-cache -t myapp .`

Specify a different Dockerfile: `docker build -f Dockerfile.prod -t myapp .`

Build from a Git repo: `docker build https://github.com/user/repo.git`

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Common flow

```
docker build -t my-app .  
docker run -p 3000:3000 my-app
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker tag

`docker tag` is a command that adds a name (tag) to an existing Docker image. It does not create a new image, it just creates another reference to the same image (think of it like giving the same photo multiple filenames).

```
docker tag <source-image>:<tag> <target-image>:<tag>
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker tag

Two common examples of this are:

- Tagging your image as the latest version: `docker tag my-app:2.0 my-app:latest`
- Tagging it for a registry (for example, Docker Hub): `docker tag myapp:latest username/myapp:1.0`

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker push

Upload a Docker image from your local machine to a remote registry (Docker Hub, GitHub Container Registry, AWS ECR...).

```
docker push <image>:<tag>
```

Make sure to tag your image correctly first!

What docker push actually does:

When you run it, Docker:

1. Checks which layers already exist in the registry
2. Uploads only missing layers (efficient, incremental upload)
3. Publishes metadata for the tag
4. Makes the image available remotely

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker push

The image name must match a registry format:

Registry	Format
Docker Hub	<code>username/repo:tag</code>
GHCR	<code>ghcr.io/user/repo:tag</code>
AWS ECR	<code>account-id.dkr.ecr.region.amazonaws.com/repo:tag</code>

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker push

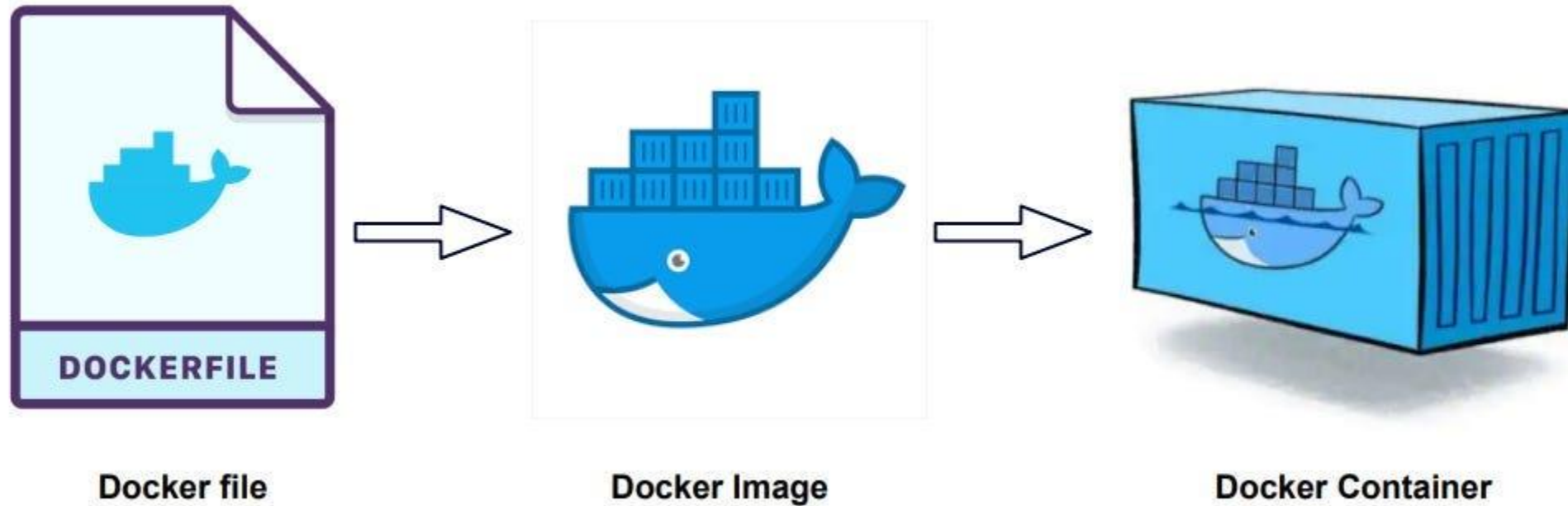
You can also push a floating tag (usually latest):

```
docker tag my-app:2.0 username/my-app:latest  
docker push username/my-app:latest
```

Be careful because of course this will overwrite what “latest” means in the registry.

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE



BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Let's have a look at some **Dockerfile** and **docker-compose.yml** files.

LAB

SETTING UP A DOCKERIZED ENVIRONMENT

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

This is for next week!

WHAT IS TRUE ABOUT DOCKER IMAGES AND CONTAINERS

- ☐ A CONTAINER IS A TEMPLATE USED TO BUILD IMAGES
- ☐ AN IMAGE IS A RUNNING INSTANCE OF A CONTAINER
- ☐ AN IMAGE IS A READ-ONLY TEMPLATE USED TO CREATE CONTAINERS
- ☐ THEY ARE IDENTICAL CONCEPTS

WHAT IS TRUE ABOUT DOCKER IMAGES AND CONTAINERS

- ☐ A CONTAINER IS A TEMPLATE USED TO BUILD IMAGES
- ☐ AN IMAGE IS A RUNNING INSTANCE OF A CONTAINER
- ☒ AN IMAGE IS A READ-ONLY TEMPLATE USED TO CREATE CONTAINERS
- ☐ THEY ARE IDENTICAL CONCEPTS

WHAT HAPPENS IF YOU REMOVE A CONTAINER WITHOUT USING A VOLUME FOR STORAGE?

- ☐ THE IMAGE IS DELETED BUT DATA REMAINS
- ☐ THE CONTAINER RESTARTS AUTOMATICALLY
- ☐ ALL DATA STORED INSIDE THE CONTAINER IS LOST
- ☐ DOCKER BACKS UP THE DATA AUTOMATICALLY

WHAT HAPPENS IF YOU REMOVE A CONTAINER WITHOUT USING A VOLUME FOR STORAGE?

- ☐ THE IMAGE IS DELETED BUT DATA REMAINS
- ☐ THE CONTAINER RESTARTS AUTOMATICALLY
- ☒ ALL DATA STORED INSIDE THE CONTAINER IS LOST
- ☐ DOCKER BACKS UP THE DATA AUTOMATICALLY

WHICH STATEMENT BEST DESCRIBES THE PURPOSE OF DOCKER NETWORKING?

- ☐ IT ENCRYPTS CONTAINER IMAGES
- ☐ IT ALLOWS CONTAINERS TO COMMUNICATE WITH EACH OTHER AND EXTERNAL SYSTEMS
- ☐ IT REPLACES THE NEED FOR ENVIRONMENT VARIABLES
- ☐ IT IS USED FOR LOGGING CONTAINER OUTPUT

WHICH STATEMENT BEST DESCRIBES THE PURPOSE OF DOCKER NETWORKING?

- ☐ IT ENCRYPTS CONTAINER IMAGES
- ☒ IT ALLOWS CONTAINERS TO COMMUNICATE WITH EACH OTHER AND EXTERNAL SYSTEMS
- ☐ IT REPLACES THE NEED FOR ENVIRONMENT VARIABLES
- ☐ IT IS USED FOR LOGGING CONTAINER OUTPUT

WHY ARE CONTAINERS CONSIDERED MORE LIGHTWEIGHT THAN VIRTUAL MACHINES?

- ❑ THEY DO NOT USE ANY CPU RESOURCES
- ❑ THEY SHARE THE HOST OPERATING SYSTEM
KERNEL
- ❑ THEY RUN WITHOUT ANY OPERATING SYSTEM
- ❑ THEY RUN FASTER THAN NATIVE
APPLICATIONS

WHY ARE CONTAINERS CONSIDERED MORE LIGHTWEIGHT THAN VIRTUAL MACHINES?

- ☐ THEY DO NOT USE ANY CPU RESOURCES
- ☒ THEY SHARE THE HOST OPERATING SYSTEM
KERNEL
- ☐ THEY RUN WITHOUT ANY OPERATING SYSTEM
- ☐ THEY RUN FASTER THAN NATIVE
APPLICATIONS

WHAT IS THE MAIN ADVANTAGE OF USING ENVIRONMENT VARIABLES IN DOCKER?

- ☐ THEY ALLOW MODIFICATION OF THE IMAGE WITHOUT REBUILDING IT
- ☐ THEY PERMANENTLY STORE DATA INSIDE CONTAINERS
- ☐ THEY REPLACE THE NEED FOR DOCKERFILES
- ☐ THEY INCREASE CONTAINER ISOLATION AT KERNEL LEVEL

WHAT IS THE MAIN ADVANTAGE OF USING ENVIRONMENT VARIABLES IN DOCKER?

- ☒ THEY ALLOW MODIFICATION OF THE IMAGE WITHOUT REBUILDING IT
- ☐ THEY PERMANENTLY STORE DATA INSIDE CONTAINERS
- ☐ THEY REPLACE THE NEED FOR DOCKERFILES
- ☐ THEY INCREASE CONTAINER ISOLATION AT KERNEL LEVEL



**Create three to five questions
about what we covered during
this first training block**

**Multiple choice questions
4 options, 1 correct answer**